Module 6 – Core Java

Introduction to Java

Theory:

o History of Java o Features of Java (Platform Independent, Object-Oriented, etc.)

History of Java

- **Java** was developed by **James Gosling** and his team at **Sun Microsystems**.
- The project started in **1991**.
- Initially, Java was called **Oak**, named after an oak tree outside Gosling's office.
- It was later renamed **Java** in **1995**.
- Java was designed to be **portable, secure, and reliable** for different devices.
- In **2010**, **Oracle Corporation** acquired Sun Microsystems and now maintains Java.

Features of Java

1. Platform Independent

- Java programs can run on **any operating system** (Windows, Linux, Mac).
- This is possible because Java uses **JVM (Java Virtual Machine)**.
- Principle: **Write Once, Run Anywhere (WORA)**.

2. Object-Oriented

- Java follows **Object-Oriented Programming (OOP)** concepts.

- Uses **classes, objects, inheritance, polymorphism, encapsulation, and abstraction**.
- Makes programs more **organized and reusable**.

3. Simple

- Java syntax is easy to learn.
- Does not support complicated features like pointers.
- Automatic **memory management** using garbage collection.

4. Secure

- Java provides strong security features.
- Uses **bytecode verification** and **JVM security**.
- Commonly used in banking and online applications.

5. Robust

- Java is strong and reliable.
- Handles errors using **exception handling**.
- Has **automatic garbage collection**.

6. Multithreaded

- Java supports **multithreading**, allowing multiple tasks to run at the same time.
- Improves performance of applications.

7. High Performance

- Uses **Just-In-Time (JIT) compiler**.
- Faster execution compared to many interpreted languages.

8. Distributed

- Java supports distributed applications.
- Used in **network-based programs**.

o Understanding JVM, JRE, and JDK

Understanding JVM, JRE, and JDK

1. JVM (Java Virtual Machine)

- JVM is a **virtual machine** that runs Java **bytecode**.
- It converts bytecode into **machine code**.
- JVM makes Java **platform independent**.
- Each operating system has its **own JVM**.
- JVM is responsible for:
    - Memory management
    - Security
    - Execution of Java programs

2. JRE (Java Runtime Environment)

- JRE provides the **environment to run** Java applications.
- It includes:
    - **JVM**
    - Core Java **libraries**
    - Supporting files
- JRE is **used by users**, not developers.

Key point: JRE = JVM + Libraries

3. JDK (Java Development Kit)

- JDK is used to **develop and run** Java programs.
- It includes:
    - **JRE**
    - Development tools like:

    - javac (compiler)
    - java (launcher)
    - javadoc
    - jar
- Required by **programmers**

Relationship Between JVM, JRE, and JDK

JDK

└── JRE

    └── JVM

Quick Comparison Table

Component	Purpose	Used By
JVM	Runs bytecode	System
JRE	Runs Java programs	Users
JDK	Develops Java programs	Developers

o Setting up the Java environment and IDE (e.g., Eclipse, IntelliJ)

Setting Up the Java Environment

Step 1: Install JDK

- Download the **Java Development Kit (JDK)** from Oracle or OpenJDK.
- Choose the version according to your system (Windows / Linux / macOS).
- Install the JDK by following the setup instructions.

Step 2: Set Environment Variables (Windows)

1. Open **System Properties** → **Advanced** → **Environment Variables**

2. Add a new **System Variable**:
    - **Variable Name:** JAVA_HOME
    - **Variable Value:** Path of JDK folder (e.g., C:\Program Files\Java\jdk)
3. Edit **Path** variable:
    - Add: C:\Program Files\Java\jdk\bin

This allows Java commands to run from the command prompt.

Step 3: Verify Java Installation

- Open **Command Prompt / Terminal**
- Type:
- java -version
- javac -version
- If Java is installed correctly, the version will be displayed.

Setting Up an IDE (Integrated Development Environment)

1. Eclipse IDE

- Download **Eclipse IDE for Java Developers**.
- Extract and run eclipse.exe.
- Choose a **workspace** location.
- Create a project:
    - File → New → Java Project
    - Write code and run using **Run → Run As → Java Application**

Advantages of Eclipse:

- Free and open source
- Good for beginners
- Large plugin support

2. IntelliJ IDEA

- Download **IntelliJ IDEA Community Edition** (free).
- Install and launch the IDE.
- Create a project:
    - New Project → Java → Select JDK
- IntelliJ handles most configurations automatically.

Advantages of IntelliJ:

- Smart code suggestions
- Powerful debugging tools
- Clean user interface

Why Use an IDE?

- Faster coding with auto-completion
- Easy debugging
- Built-in compiler and error checking
- Better project management

Summary

- **JDK** is required to write and run Java programs.
- **IDE** makes Java development easier and faster.
- **Eclipse** and **IntelliJ IDEA** are popular Java IDEs.

o Java Program Structure (Packages, Classes, Methods)

2. Data Types, Variables, and Operators

• Theory:

o Primitive Data Types in Java (int, float, char, etc.)

Primitive Data Types in Java

Java has **8 primitive data types**. They store **simple values** and are not objects.

1. Integer Data Types

Data Type	Size	Description
byte	1 byte	Very small integers
short	2 bytes	Small integers
int	4 bytes	Most commonly used integer
long	8 bytes	Very large integers

2. Floating-Point Data Types

Data Type	Size	Description
float	4 bytes	Decimal numbers (single precision)
double	8 bytes	Decimal numbers (double precision)

3. Character Data Type

Data Type	Size	Description
char	2 bytes	Stores a single character

o Variable Declaration and Initialization

2. Variable Initialization

- Initialization means **assigning a value** to a variable.
- Syntax:

variableName = value;

Example:

number = 10;

3. Declaration and Initialization Together

- Both can be done in a single line.

Example:

int marks = 85;

Rules for Naming Variables

- Must start with a **letter, _, or $**
- Cannot start with a number
- No spaces allowed
- Cannot use Java **keywords**
- Variable names are **case-sensitive**

o Operators: Arithmetic, Relational, Logical, Assignment, Unary, and Bitwise

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

Operator	Meaning
%	Modulus (remainder)

Example:

```
int a = 10, b = 3;
System.out.println(a + b);  // 13
System.out.println(a % b);  // 1
```

2. Relational Operators

Used to **compare two values**. Result is true or false.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
int x = 5, y = 10;
System.out.println(x < y); // true
```

3. Logical Operators

Used to combine **boolean expressions**.

Operator	Meaning
&&	Logical AND

Operator	Meaning
`	
!	Logical NOT

Example:

```
boolean a = true, b = false;
System.out.println(a && b); // false
```

4. Assignment Operators

Used to **assign values** to variables.

Operator	Example
=	a = 5
+=	a += 3
-=	a -= 2
*=	a *= 4
/=	a /= 2

Example:

```
int a = 5;
a += 3;  // a =
```

5. Unary Operators

Operate on **a single operand**.

Operator	Meaning
++	Increment
--	Decrement
+	Unary plus

Operator	Meaning
-	Unary minus
!	Logical NOT

Example:

```
int a = 5;
a++;  // a becomes 6
```

6. Bitwise Operators

Used for **bit-level operations**.

Operator	Meaning
&	Bitwise AND
`	`
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

Example:

```
int a = 5, b = 3;
System.out.println(a & b); // 1
```

o Type Conversion and Type Casting

Type Conversion and Type Casting

1. Type Conversion (Widening / Implicit Casting)

- Done **automatically** by Java.
- Converts **smaller data type to larger data type**.
- No data loss.

Order:

byte → short → int → long → float → double

Example:

```
int a = 10;
double b = a;  // automatic conversion
```

2. Type Casting (Narrowing / Explicit Casting)

- Done **manually** by the programmer.
- Converts **larger data type to smaller data type**.
- May cause **data loss**.

Syntax:

```
(dataType) value
```

Example:

```
double a = 9.7;
int b = (int) a;  // b = 9
```

Difference Between Type Conversion and Type Casting

Type Conversion	Type Casting
Automatic	Manual
Smaller → Larger	Larger → Smaller

Type Conversion	Type Casting
No data loss	Data loss possible

3. Control Flow Statements

- Theory:

o If-Else Statements

1. If–Else Statements

Used to **make decisions** based on conditions.

Syntax

```
if (condition) {
   // code if condition is true
} else {
   // code if condition is false
}
```

Example

```
int age = 18;

if (age >= 18) {
   System.out.println("Eligible to vote");
} else {
   System.out.println("Not eligible to vote");
}
```

Types

- Simple if
- if–else
- else if ladder
- Nested if

o Switch Case Statements

2. Switch Case Statements

Used when there are **multiple choices** based on one variable.

Syntax

```
switch (expression) {
   case value1:
      // code
      break;
   case value2:
      // code
      break;
   default:
      // code
}
```

Example

```
int day = 2;


switch (day) {
   case 1:
      System.out.println("Monday");
      break;
   case 2:
      System.out.println("Tuesday");
      break;
   default:
```

```
        System.out.println("Invalid day");
}
```

o Loops (For, While, Do-While)

Loops are used to **repeat a block of code**.

A. For Loop

Used when the number of iterations is **known**.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

B. While Loop

Checks condition **before** execution.
Used when iterations are **not fixed**.

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

C. Do–While Loop

Executes **at least once**, condition checked **after** execution.

```
int i = 1;
do {
    System.out.println(i);
    i++;
```

```
} while (i <= 5);
```

o Break and Continue Keywords

Break

- Used to **exit a loop or switch** immediately.

```
for (int i = 1; i <= 5; i++) {
   if (i == 3)
      break;
   System.out.println(i);
}
```

Output: 1 2

Continue

- Skips the **current iteration** and moves to the next one.

```
for (int i = 1; i <= 5; i++) {
   if (i == 3)
      continue;
   System.out.println(i);
}
```

Output: 1 2 4 5

4. Classes and Objects

- Theory:

o Defining a Class and Object in Java

Class

- A **class** is a **blueprint or template** used to create objects.
- It defines **properties (variables)** and **behaviors (methods)**.
- A class does not occupy memory until an object is created.

Object

- An **object** is an **instance of a class**.
- It represents a real-world entity.
- Memory is allocated only when an object is created.

Example (theory):
Class = Student
Object = A particular student like Rahul or Anita

o Constructors and Overloading

Constructors in Java

- A **constructor** is a special method used to **initialize objects**.
- Constructor name is **same as the class name**.
- It has **no return type**.
- It is automatically called when an object is created.

Types of Constructors

1. **Default Constructor**
    - Takes no parameters
    - Initializes objects with default values

2. **Parameterized Constructor**
    - Takes parameters
    - Initializes objects with user-defined values

Constructor Overloading

- **Constructor overloading** means having **more than one constructor** in the same class.
- Constructors must differ in **number or type of parameters**.
- It allows creating objects in **different ways**.
- It is an example of **compile-time polymorphism**.

o Object Creation, Accessing Members of the Class

- Objects are created using the **new keyword**.
- When an object is created:
    - Memory is allocated
    - Constructor is called automatically
- Multiple objects can be created from the same class.

Accessing Members of a Class

- Class members include:
    - **Data members (variables)**
    - **Member functions (methods)**
- Members are accessed using the **dot (.) operator**.
- Access depends on the **access specifier** (public, private, etc.).

o this Keyword

this Keyword in Java

- The **this keyword** refers to the **current object**.
- It is mainly used to:
    - Differentiate between **instance variables and parameters**
    - Refer to the current class object
    - Call another constructor in the same class
- It improves **code clarity** and avoids ambiguity.

5. Methods in Java

- Theory:

o Defining Methods

A **method** is a block of code that performs a **specific task**.

Methods help in **code reusability** and **modular programming**.

A method is defined inside a **class**.

o Method Parameters and Return Types

Method Parameters

- Parameters are used to **pass values** to a method.
- They act as **input** for the method.
- A method can have:
    - No parameters
    - One or more parameters

Return Type

- The **return type** specifies the type of value returned by a method.
- If a method does not return any value, it uses the **void** return type.
- A method can return **only one value** at a time.

o Method Overloading

- **Method overloading** means defining **multiple methods with the same name** in the same class.
- Methods must differ in:
    - Number of parameters, or
    - Type of parameters, or
    - Order of parameters
- Return type alone **cannot** differentiate overloaded methods.
- It is an example of **compile-time polymorphism**.

o Static Methods and Variables

Static Variables

- Declared using the **static** keyword.
- Shared among **all objects** of the class.
- Memory is allocated **only once**.
- Used for **common data**.
- Example (theory): College name shared by all students.

Static Methods

- Declared using the **static** keyword.
- Belong to the **class**, not to objects.
- Can be called **without creating an object**.
- Can access **only static variables directly**.

The main() method is a **static method**.

6. Object-Oriented Programming (OOPs) Concepts

- Theory:

o Basics of OOP: Encapsulation, Inheritance, Polymorphism, Abstraction

1. Encapsulation

- **Encapsulation** means **wrapping data and methods together** into a single unit (class).
- It protects data from **unauthorized access**.
- Achieved using **access modifiers** like private, public, and protected.

Advantages

- Improves **data security**
- Makes code **maintainable**
- Supports **data hiding**

Polymorphism means **one name, many forms**.

- Same method behaves **differently** based on the object.

Types of Polymorphism

1. **Compile-time Polymorphism** → Method Overloading
2. **Run-time Polymorphism** → Method Overriding

4. Abstraction

- **Abstraction** means **hiding implementation details** and showing only essential features.
- Achieved using:
    - **Abstract classes**
    - **Interfaces**

Advantages

- Reduces complexity
- Improves security

- Helps in managing large programs

o Inheritance: Single, Multilevel, Hierarchical

2. Inheritance

- **Inheritance** allows one class to **acquire properties and methods of another class**.
- The existing class is called **Parent/Superclass**.
- The new class is called **Child/Subclass**.
- Promotes **code reusability**.

Types of Inheritance in Java

A. Single Inheritance

- One child class inherits from **one parent class**.
- Example (theory):
  Class B inherits from Class A

B. Multilevel Inheritance

- A class is derived from another derived class.
- Forms a **chain of inheritance**.

Example (theory):
Class C inherits from Class B, and Class B inherits from Class A.

C. Hierarchical Inheritance

- **Multiple child classes** inherit from **one parent class**.

Example (theory):
Class B and Class C inherit from Class A.

Advantages of Inheritance

- Reduces **code duplication**
- Improves **code structure**
- Supports **extensibility**

o Method Overriding and Dynamic Method Dispatch

Method Overriding

- **Method overriding** occurs when a child class provides its **own implementation** of a method already defined in the parent class.
- Method name, return type, and parameters must be **same**.
- Happens during **run-time**.

Key Points

- Requires **inheritance**
- Supports **run-time polymorphism**

Dynamic Method Dispatch

- Dynamic Method Dispatch is the process where the **method call is resolved at run-time**.
- The method of the **object type**, not the reference type, is executed.
- Implemented using **method overriding**.

7. Constructors and Destructors

- Theory:

o Constructor Types (Default, Parameterized)

A **constructor** is a special method used to **initialize an object**.
It is automatically called when an object is created.

1. Default Constructor

- A constructor **without parameters**.
- Initializes objects with **default values**.
- If no constructor is defined, Java provides a **default constructor** automatically. **Use:** To create objects with initial default data.

2. Parameterized Constructor

- A constructor that **accepts parameters**.
- Used to initialize objects with **user-defined values**.
- Provides flexibility during object creation.

Use: To create objects with specific values.

o Copy Constructor (Emulated in Java)

Copy Constructor (Emulated in Java)

- Java does **not have a built-in copy constructor** like C++.
- Copy constructor behavior is **emulated** by:
    - Passing an object as a parameter to a constructor.
- It creates a **new object by copying values** from an existing object.

Purpose

- To duplicate an object
- To avoid reference sharing

- To create independent copies

o Constructor Overloading

- **Constructor overloading** means defining **multiple constructors** in the same class.
- Constructors must differ in:
    - Number of parameters
    - Type of parameters
- Enables creating objects in **different ways**.
- It is an example of **compile-time polymorphism**.

Advantages

- Improves flexibility
- Makes object creation easier
- Enhances readability

o Object Life Cycle and Garbage Collection

The **object life cycle** describes the stages an object goes through in memory.

Stages of Object Life Cycle

1. **Creation**
    - Object is created using the new keyword.
    - Memory is allocated in the heap.
2. **Usage**
    - Object is used to access data and methods.
3. **Eligibility for Garbage Collection**
    - Object is no longer referenced.
    - Becomes unused or unreachable.
4. **Destruction**

Garbage Collector removes the object from memory.

- **Garbage Collection** is the process of **automatic memory management**.
- Java removes objects that are **no longer in use**.
- Performed by the **Garbage Collector**.
- Helps prevent **memory leaks**.

Advantages of Garbage Collection

- Automatic memory cleanup
- Improves application performance
- Reduces programmer effort

8. Arrays and Strings

- Theory:

o One-Dimensional and Multidimensional Arrays

Array (Basic Concept)

- An **array** is a collection of **similar data types** stored under a single name.
- Array elements are stored in **contiguous memory locations**.
- Each element is accessed using an **index number** (starts from 0).

One-Dimensional Array

- A **one-dimensional array** stores data in a **linear form**.
- Used to store a list of values of the same type.

Example (theory):
Marks of students in a single subject.

Advantages

- Easy to use
- Fast access to elements

Multidimensional Array

- A **multidimensional array** stores data in **row and column format**.
- Commonly used as a **2-D array (matrix)**.

Example (theory):
Marks of students in multiple subjects.

Advantages

- Organized data storage
- Useful for tables and matrices

o

o Array of Objects o String Methods (length, charAt, substring, etc.)

ength()

- Returns the **number of characters** in a string.

charAt(index)

- Returns the **character at a specific position**.

substring()

- Extracts a **part of the string**.

equals()

- Compares **content** of two strings.

toUpperCase() / toLowerCase()

- Converts string to **uppercase or lowercase**.

trim()

- Removes **leading and trailing spaces**.

String Handling in Java: String Class, StringBuffer, StringBuilder

1. String Class

- String represents a **sequence of characters**.
- Strings are **immutable** (cannot be changed once created).
- Any modification creates a **new object**.

Advantages

- Secure
- Thread-safe

Used when string data does not change frequently.

2. StringBuffer

- StringBuffer is **mutable** (can be modified).
- It is **thread-safe**.
- Slower than StringBuilder.
- Used in **multi-threaded** environments.

3. StringBuilder

- StringBuilder is **mutable**.
- It is **not thread-safe**.
- Faster than StringBuffer.
- Used when performance is important and only **single thread** is involved.

Difference Between String, StringBuffer, and StringBuilder

Feature	**String**	**StringBuffer**	**StringBuilder**
Mutability	Immutable	Mutable	Mutable

Feature	String	StringBuffer	StringBuilder
Thread-safe	Yes	Yes	No
Performance	Slow	Medium	Fast

9. Inheritance and Polymorphism

- Theory:

o Inheritance Types and Benefits

Meaning

- **Inheritance** is a mechanism where one class **acquires the properties and methods** of another class.
- The existing class is called the **Superclass (Parent)**.
- The derived class is called the **Subclass (Child)**.
- It supports **code reusability**.

Types of Inheritance in Java

a) Single Inheritance

- One child class inherits from **one parent class**.

b) Multilevel Inheritance

- A class inherits from another derived class, forming a **chain**.

c) Hierarchical Inheritance

- Multiple child classes inherit from **one parent class**.
- **Note:** Java does not support multiple inheritance using classes (to avoid ambiguity).

Benefits of Inheritance

- Reduces **code duplication**
- Improves **code reusability**
- Enhances **maintainability**
- Supports **polymorphism**
- Improves **program structure**

o Method Overriding

Meaning

- **Method overriding** occurs when a subclass provides its **own implementation** of a method already defined in the superclass.
- Method name, parameters, and return type must be **the same**.
- It occurs at **run-time**.

Purpose

- To provide **specific behavior** in the child class.
- Supports **run-time polymorphism**.

o Dynamic Binding (Run-Time Polymorphism)

Meaning

- **Dynamic binding** means the **method call is resolved at run-time**, not at compile time.
- The method executed depends on the **type of object**, not the reference variable.
- Implemented using **method overriding**.

Importance

- Makes programs **flexible**
- Allows **one interface, multiple implementations**
- Improves **extensibility**

o Super Keyword and Method Hiding

Meaning

- The **super keyword** is used to refer to the **immediate parent class object**.

- It helps avoid ambiguity between parent and child class members.

Uses

- Access parent class variables
- Call parent class methods
- Invoke parent class constructor

5. Method Hiding

Meaning

- **Method hiding** occurs when a subclass defines a **static method** with the same name and signature as a static method in the superclass.
- Unlike overriding, method hiding is resolved at **compile time**.

10. Interfaces and Abstract Classes

- Theory:

o Abstract Classes and Methods

Abstract Class

- An **abstract class** is a class that is **declared using the abstract keyword**.
- It **cannot be instantiated** (objects cannot be created).
- It may contain:
    - **Abstract methods** (without body)
    - **Concrete methods** (with body)
- Used to provide a **base structure** for subclasses.

Purpose: To achieve **partial abstraction**.

Abstract Method

- An **abstract method** is a method **without implementation**.
- Declared using the abstract keyword.
- Must be **implemented by the subclass**.

Abstract methods define **what to do**, not **how to do**.

Advantages of Abstract Classes

- Supports code **reusability**
- Ensures **method implementation** in child classes
- Helps in **design consistency**

o Interfaces: Multiple Inheritance in Java

Meaning

- An **interface** is a blueprint of a class.

- It contains:
    - Abstract methods
    - Static constants
- All methods in an interface are **public and abstract by default**.
- Interfaces provide **100% abstraction**.

Multiple Inheritance in Java (Using Interfaces)

- Java does **not support multiple inheritance using classes** (to avoid ambiguity).
- Java **supports multiple inheritance using interfaces**.
- A class can **implement more than one interface**.

This avoids problems like **diamond ambiguity**.

Why Java Uses Interfaces for Multiple Inheritance

- Interfaces contain **only method declarations**
- No implementation conflict
- Ensures clear method definitions

Implementing Multiple Interfaces

- A class can implement **multiple interfaces at the same time**.
- The class must **provide implementations for all methods** of all interfaces.
- This allows a class to inherit behavior from **multiple sources**.

Used when a class needs to follow **multiple contracts**.

Difference Between Abstract Class and Interface

Abstract Class	Interface
Can have abstract & concrete methods	Only abstract methods
Supports partial abstraction	Supports full abstraction
Can have constructors	Cannot have constructors
Uses extends	Uses implements

o Implementing Multiple Interfaces

```
interface Printable {
   void print();
}

interface Showable {
   void show();
}
class Demo implements Printable, Showable {

   public void print() {
      System.out.println("Printing...");
   }

   public void show() {
      System.out.println("Showing...");
   }
}
public class Main {
   public static void main(String[] args) {
```

```
        Demo obj = new Demo();
        obj.print();
        obj.show();
    }
}
```

11. Packages and Access Modifiers

- Theory:

o Java Packages: Built-in and User-Defined Packages

Meaning of Package

- A **package** is a group of **related classes and interfaces**.
- Packages help in **organizing large programs**.
- They avoid **name conflicts** and improve **code maintenance**.

Types of Packages in Java

1. Built-in Packages

- Provided by Java API.
- Contain predefined classes and interfaces.
- Commonly used built-in packages:
    - java.lang → basic classes (String, Math, System)
    - java.util → utility classes (Scanner, ArrayList)
    - java.io → input/output operations
    - java.sql → database connectivity
    - java.lang is imported **automatically**.

2. User-Defined Packages

- Created by programmers.
- Used to group related classes.
- Helps in **better project structure** and reuse.

o Access Modifiers: Private, Default, Protected, Public

1. Private

- Accessible **only within the same class**.
- Provides **maximum data hiding**.
- Not accessible outside the class.

Used for sensitive data.

2. Default (No Modifier)

- Accessible **within the same package only**.
- Also called **package-private**.

Used when access is limited to a package.

3. Protected

- Accessible:
    - Within the same package
    - In subclasses (even outside the package)

Used in inheritance.

4. Public

- Accessible **from anywhere**.
- Least restrictive access level.

Used for main methods and APIs.

Access Modifiers Summary Table

Modifier	Same Class	Same Package	Subclass	Everywhere
Private	✓	✕	✕	✕

Modifier	Same Class	Same Package	Subclass	Everywhere
Default	✓	✓	✕	✕
Protected	✓	✓	✓	✕
Public	✓	✓	✓	✓

o Importing Packages and Classpath

- The **import statement** is used to access classes from a package.
- It improves readability and avoids using full class names.

Types of Import

1. **Single class import**
2. **Whole package import**
3. Without import, fully qualified class names must be used.

Classpath in Java

Meaning

- **Classpath** is the path where Java looks for **classes and packages**.
- It tells the JVM where to find .class files.

Purpose

- Required to run Java programs successfully.
- Helps in locating user-defined and external libraries.

12. Exception Handling

- Theory:

    1. **Types of Exceptions in Java**
    2. **Checked Exceptions**

- Checked **at compile time**
- Must be **handled using try–catch** or **declared using throws**
- Usually caused by external factors

Examples:

- IOException
- SQLException
- FileNotFoundException

```
import java.io.*;


class Test {
   public static void main(String[] args) throws IOException {
      FileReader fr = new FileReader("data.txt");
   }
}
```

Unchecked Exceptions

- Checked **at runtime**
- Not compulsory to handle
- Usually caused by programming errors

Examples:

- NullPointerException
- ArithmeticException

- ArrayIndexOutOfBoundsException

```
class Test {
   public static void main(String[] args) {
      int a = 10 / 0; // ArithmeticException
   }
}
```

2. try, catch, finally

- **try**
  - Code that may cause an exception is written here
- **catch**
  - Handles the exception
- **finally**
  - Executes **whether an exception occurs or not**
  - Used for cleanup (closing files, database connections)

```
class Test {
   public static void main(String[] args) {
      try {
         int a = 10 / 0;
      } catch (ArithmeticException e) {
         System.out.println("Error: Division by zero");
      } finally {
         System.out.println("This always executes");
      }
   }
}
```

3. throw Keyword

- Used to **explicitly throw an exception**
- Used inside a method or block

```
class Test {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Not eligible to vote");
        }
        System.out.println("Eligible to vote");
    }

    public static void main(String[] args) {
        checkAge(16);
    }
}
```

4. throws Keyword

- Used in **method declaration**
- Tells the caller that the method may throw an exception

```
import java.io.*;

class Test {
    static void readFile() throws IOException {
        FileReader fr = new FileReader("file.txt");
    }
```

```
    public static void main(String[] args) throws IOException {
        readFile();
    }
}
```

5. Custom Exception Classes

- User-defined exceptions
- Created by extending Exception or RuntimeException

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

class Test {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above");
        }
        System.out.println("Valid age");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
```

```
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

13. Multithreading

- Theory:

o Introduction to Threads

Meaning

- A **thread** is a **lightweight unit of execution** within a program.
- A single program can have **multiple threads running simultaneously** (multithreading).
- Threads share the **same memory space**, making communication faster.

Advantages of Threads

- Better **CPU utilization**
- **Faster execution** of tasks
- Supports **concurrent programming**
- Useful for **real-time applications**

o Creating Threads by Extending Thread Class or Implementing Runnable Interface

1. By Extending Thread Class

- Create a subclass of Thread and **override the run() method**.
- Call start() to begin execution.

2. By Implementing Runnable Interface

- Implement Runnable and **override run() method**.
- Pass the Runnable object to a Thread object and call start().

o Thread Life Cycle

A thread goes through **various states**:

State	Description
New / Created	Thread object is created but not started
Runnable	Thread is ready to run, waiting for CPU time
Running	Thread is executing its run() method
Waiting / Blocked	Thread is waiting for some event or resource
Timed Waiting	Thread waits for a specified time
Terminated / Dead	Thread has finished execution

incomplete

o Synchronization and Inter-thread Communication

14. File Handling

• Theory:

o Introduction to File I/O in Java (java.io package)

Meaning

- File I/O (Input/Output) allows Java programs to **read from and write to files**.
- Java provides the **java.io package** for file handling.
- Files can be **text files or binary files**.

Advantages

- Store data permanently
- Exchange data between programs
- Useful for **large data processing**

o FileReader and FileWriter Classes

1. FileReader

- Used to **read character-based files**.
- Reads data **one character at a time**.
- Suitable for **text files**.

2. FileWriter

- Used to **write character-based files**.
- Writes data **one character at a time**.
- Can **overwrite** or **append** data in files.

o BufferedReader and BufferedWriter

BufferedReader

- Wraps around a **FileReader**.
- Reads data **line by line** using readLine().
- More **efficient** than FileReader.

BufferedWriter

- Wraps around a **FileWriter**.
- Writes data **efficiently** using write() and newLine().
- Reduces **I/O operations** for better performance.

o Serialization and Deserialization

Serialization

- Process of **converting an object into a byte stream**.
- Allows objects to be **saved to files or transmitted** over networks.
- Classes must **implement Serializable interface**.

Deserialization

- Process of **converting a byte stream back into an object**.
- Restores the object with its **original state**.

16. Java Input/Output (I/O)

• Theory:

o Streams in Java (InputStream, OutputStream)

Meaning

- A **stream** is a **sequence of data** (bytes or characters) flowing from a source to a destination.
- Java uses streams for **reading and writing data** from/to files, memory, or network connections.
- Part of the **java.io package**.

Types of Streams

Stream Type	Description	Example Classes
Byte Stream	Reads/writes **one byte at a time**	InputStream, OutputStream, FileInputStream, FileOutputStream
Character Stream	Reads/writes **characters**	Reader, Writer, FileReader, FileWriter

o Reading and Writing Data Using Streams

Reading

- Use InputStream subclasses to **read bytes from a source**.
- Can read **one byte at a time** or into a **byte array**.
- Buffered streams improve **efficiency**.

Writing

- Use OutputStream subclasses to **write bytes to a destination**.
- Can write **one byte at a time** or from a **byte array**.

- Buffered streams reduce the number of **I/O operations**.

o Handling File I/O Operations

Key Steps

1. **Create a stream object** linked to a file.
2. **Perform read/write operations**.
3. **Close the stream** to release system resources.
4. **Handle exceptions** using try-catch or throws.